

Management Report: Optimisation of Operator Schedules at Lancaster University

Executive Summary

This report provides the findings and methods used to optimise the scheduling of student operators (Bachelor's and Master's) at Lancaster University's computer facility's mainframe. The project involved reimplementing an existing linear programming model in Python and producing separate models that involve fairness constraints and incorporate individual skills of the student operators. It also involved providing a user-friendly interface for Eleanor Mitchell, the director of the computer facility, that allows you to choose which of the three models you would like to see the timetable for. The program also visualises the results as a bar chart showing the Total Hour Distribution Per Week for each Analysis.

The total cost for Analysis 1 (The reimplementing of the original model) showed a total cost of £1755.00, with the maximum hours assigned to an operator being 20. The total cost for Analysis 2 (Introducing fairness constraints) was £1786.59, which perfectly matched the placed budget limit on analysis 2 (an increase of cost of no more than 1.8%). The maximum hours assigned to an operator were 13.62. The total cost of Analysis 3 (Introducing skill-based requirements) was £1762.00, and the maximum hours assigned to an operator was 20. Below is a table summary of that information.

Key Findings

Experiment	Explanation	Total Cost	Cost increase	Maximum hours assigned to an operator
Analysis 1	Reimplementation of original model	£1755.00	+£0	20.00
Analysis 2	Introduction of fairness constraint	£1786.59	+£31.59	13.62
Analysis 3	Introduction of skill-based requirements	£1762.00	+£7.00	20.00

Introduction

Background

Lancaster University has a mainframe computer, which needs to be managed every day from 8 am to 10 pm. It has employed six student operators to manage it from Monday to Friday. Each operator has different wage rates, skills, daily availabilities, and, depending on their course, different minimal working requirements. The director, Eleanor Mitchell, wants to optimise the schedule to minimise costs, ensure fairness and meet operational requirements.

Objectives

The objectives of this project were as follows:

1. Reimplement the existing linear programming model in Python and verify its results.
2. Create a secondary model to include fairness constraints, at no higher cost than 1.8% of the cost of the original model.
3. Create a tertiary model which implements the fairness constraint as well as incorporates skill-based requirements to ensure sufficient daily coverage for programming and troubleshooting tasks.
4. Create a simple user interface that will allow Eleanor Mitchell to navigate between these three models, as well as show the total cost and a bar chart of the distributed working hours between operators.

Methodology

Reimplementation of the Original Model (Analysis 1)

The original linear programming model was reimplemented using the `pulp` library in Python. The model minimises the total cost of operator wages while satisfying the following constraints:

1. Operators cannot be assigned more than their daily availability.
2. Every day, a total of 14 hours of work must be completed by the operators.
3. Bachelor's students must work a minimum of 8 hours weekly.
4. Master's students must work a minimum of 7 hours weekly.
5. Negative work hours are not accepted by the program.

The objective function used in this model is:

$$\text{Minimise } \sum_{o \in O} \sum_{d \in D} c_o \cdot x_{od}$$

Where c_o is the wage rate for operator o , and x_{od} represents the hours assigned to operator o on day d .

Fairness Constraints (Analysis 2)

In the solution above, there is a notable unfairness to the distribution of hours between the operators. To rectify this, the model requirements were extended to minimise the maximum hours assigned to any particular operator, while making sure not to increase the total cost by more than 1.8% of the original cost. Along with the constraints in Analysis 1, we add an extra constraint: minimising maximum hours.

$$\sum_{d \in D} x_{od} \leq \text{maxhours} \quad \forall_o \in O$$

And the objective function used in this model is

$$\sum_{o \in O} \sum_{d \in D} c_o \cdot x_{od} \leq 1755 \times 1.018$$

Skill-Based Requirements (Analysis 3)

The third model was extended even further to ensure that at least 6 hours of work per day were dedicated to one of the two specific skill types, programming and troubleshooting. These additional constraints are as follows.

$$\sum_{o \in S_p} x_{od} \geq 6 \quad \forall d \in D$$

$$\sum_{o \in S_t} x_{od} \geq 6 \quad \forall d \in D$$

Where S_p and S_t are the sets of operators with Programming and Troubleshooting skills, respectively.

Python Code

All of the above mathematical constraints were written in Python in each respective Python function for each analysis, before employing the pulp library in order to easily solve these linear programming equations. The timetables for each of these analysis was easily built using the python library 'tabulate, and are integrated with the python functions of each of the analysis'.

A separate function was used to easily create the bar chart for each of the analysis, employing the python library 'matplotlib.pyplot' to do so. The program also uses an if/else statement in order to display "No optimal solution found" if the constraints are too limiting.

The given data for this project, such as wage rates, availability, minimum hours each operator should work, and individual skills of each student, have all been stored as dictionaries at the start of the code. Operator names and days of the week have been stored as lists.

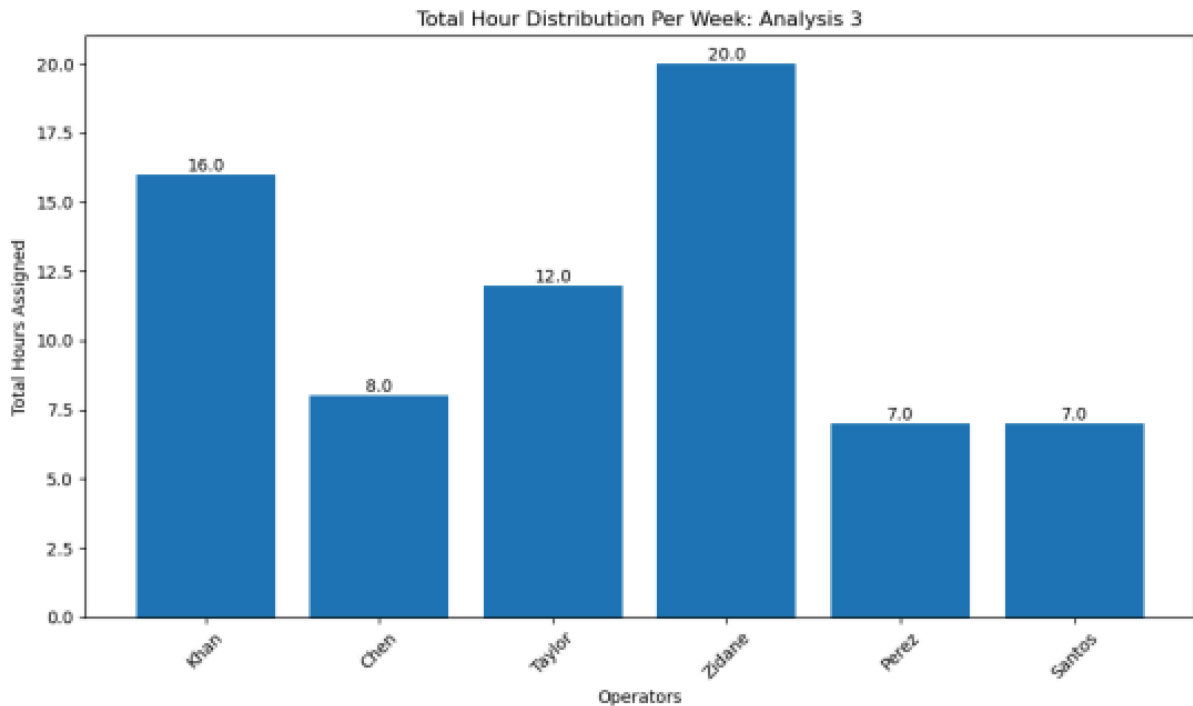
The used interface was built to display all 3 possible options related to the possible analysis choices, as well as a fourth option, 'Exit', in order to exit the program. If the user does not enter the required input for one of these options, the program will display "Invalid choice. Please try again."

Results and Findings

Analysis 1: Original Model

The reimplementation confirmed the result of the original total cost of £1755. As shown below, the schedule met all the constraints, with some operators, such as Zidane and Taylor being assigned significantly more hours than others.

Optimal timetable for Analysis 1:						
Operator	Monday	Tuesday	Wednesday	Thursday	Friday	Total
Khan	4.00	0.00	5.00	0.00	0.00	9.00
Chen	0.00	2.00	0.00	6.00	0.00	8.00
Taylor	4.00	7.00	4.00	0.00	4.00	19.00
Zidane	5.00	5.00	5.00	0.00	5.00	20.00
Perez	0.00	0.00	0.00	7.00	0.00	7.00
Santos	1.00	0.00	0.00	1.00	5.00	7.00
Total cost: £1755.00						



Conclusions and Recommendations

Conclusion

The reimplemented model fully confirmed the original results produced by the director Eleanor Mitchel, which validates the implementation of the Python code. The subsequently introduced fairness constraints successfully reduced the disparities in the workload, with a minimal cost increase. In the third analysis, the skill-based requirements were met, ensuring adequate daily coverage for critical tasks, as well as only increasing the original cost by £7 while still managing to disperse the workload more evenly between the operators than in Analysis 1.

Recommendations

1. Further Fairness Metric: Explore the use of additional metrics, such as workload variance, to distribute the workload more evenly
2. Increase the detail of the timetable to be hourly, instead of daily, as well as take the availability of each operator by the hours daily available.
3. Incorporate more skill categories if the operational requirements expand.

Model Limitations

1. The model assumes fixed availability and wage rates. Changes in these parameters would require manual updates.
2. The skill constraints may not be feasible if operator availability is highly restricted.